Reviewer Pack — Minimal Rank of p-adic Logarithmic Potential and Tseitin Res…

Entry 7ba1444f0780 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 19:20:15 UTC

Minimal Rank of p-adic Logarithmic Potential and Tseitin Resolution Length

Entry ID: `7ba1444f0780` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 19:20:15 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (p-adic logarithm) **Field B** (complexity object): Complexity Theory: Tseitin Resolution Complexity

Statement:

> {'stmt1': 'For every CNF formula F with n variables, let ϕ_F be the p-adic logarithmic potential of the indicator function of F, defined as the p-adic logarithm of the number of satisfying assignments of F.', 'stmt2': 'Then, the minimal rank r_F of ϕ_F is a lower bound on the Tseitin resolution length of F, i.e., r_F ≤ τ(F), where τ(F) is the minimum depth of a resolution proof for F.', 'stmt3': 'Furthermore, there exists an absolute constant c_p such that r_F ≥ c_pτ(F).'}

Rationale (proposer's reasoning):

> {'ration1': 'The p-adic logarithmic potential provides a natural measure of complexity in the context of p-adic numbers, which could offer insights into the structure of Boolean functions.', 'ration2': 'The Tseitin resolution length is a fundamental measure of the complexity of a CNF formula, and this conjecture proposes a direct connection between these two concepts.', 'ration3': 'If true, this connection would suggest that p-adic analysis could be used to develop new algorithms for solving NP-complete problems.'}

Taxonomy category: `TROPICAL_FOURIER_ANALYSIS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `28f515704ce99ddb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all generated CNF formulas F, the ratio of minimal rank r_F to Tseitin resolution length τ(F) is within a factor of c_p ± 5% and the mean difference between r_F and c_pτ(F) is less than 1. The conjecture is falsified if this ratio is outside this range or if any seed produces a discrepancy greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `p-adic logarithm AND Tseitin resolution length` - `minimal rank p-adic potential AND CNF formula complexity` - `c_p constant lower bound p-adic log potential resolution depth`

Top relevant hits considered: - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P-Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/hep-th/9410058v3] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [http://arxiv.org/abs/1510.02114v3] The p-adic Gross-Zagier formula on Shimura curves - [http://arxiv.org/abs/1802.05923v3] Small totally p-adic algebraic numbers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import itertools
from fractions import Fraction

def p_adic_log(x, p):
    if x <= 0:
        return None
    count = 0
    while x % p == 0:
        x //= p
        count += 1
    return count

def generate_cnf(n, alpha):
    clauses = []
    for _ in range(int(alpha * n)):
        clause = random.sample(range(1, n + 1), random.randint(1, n))
        if random.choice([True, False]):
            clause = [-x for x in clause]
        clauses.append(clause)
    return clauses

def tseitin_resolution_length(cnf):
```

```
    # Placeholder function to compute Tseitin resolution length
    # This is a stub and should be replaced with an actual implementation
    return len(cnf) + 10  # Example value for demonstration purposes

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    alpha = random.uniform(0.2, 0.8)
    cnf = generate_cnf(n, alpha)

    tau_F = tseitin_resolution_length(cnf)

    phi_F = p_adic_log(sum(1 for assignment in itertools.product([0, 1], repeat=n) if all(all(assi

    if phi_F is None:
        return {
            "metric_name": "minimal_rank",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "p_adic_log returned None"
        }

    r_F = phi_F

    if tau_F == 0:
        return {
            "metric_name": "minimal_rank",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Tseitin resolution length is zero"
        }

    ratio = r_F / tau_F
    mean_diff = abs(r_F - Fraction(2, 3) * tau_F)

    return {
        "metric_name": "minimal_rank",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": 0.95 <= ratio <= 1.05 and mean_diff < 1,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 31))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
```

```
    std_dev = (sum((r["metric_value"] - mean_ratio) ** 2 for r in results if r["metric_value"] is
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["metric_value"] is not None for r in results):
        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fr
        else:
            print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
    elif any(r["metric_value"] is None for r in results):
        print("RESULT: INCONCLUSIVE some trials produced None")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"p_adic_log returned None\" first_failing_seed={
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fee3099.py", line 87, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fee3099.py", line 48, in run_trial
   phi_F = p_adic_log(sum(1 for assignment in itertools.product([0, 1], repeat=n) if all(all(assignme
1] == literal % 2 for literal in clause) for clause in cnf)), 2)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fee3099.py", line 48, in <genexpr>
   phi_F = p_adic_log(sum(1 for assignment in itertools.product([0, 1], repeat=n) if all(all(assignme
1] == literal % 2 for literal in clause) for clause in cnf)), 2)
                                                                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fee3099.py", line 48, in <genexpr>
   phi_F = p_adic_log(sum(1 for assignment in itertools.product([0, 1], repeat=n) if all(all(assignme
1] == literal % 2 for literal in clause) for clause in cnf)), 2)
                                                                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9fee3099.py", line 48, in <genexpr>
   phi_F = p_adic_log(sum(1 for assignment in itertools.product([0, 1], repeat=n) if all(all(assignme
1] == literal % 2 for literal in clause) for clause in cnf)), 2)
                                                                                          ^^^
NameError: name 'var' is not defined. Did you mean: 'vars'?
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

The test crashed due to a NameError, which indicates that the code was not correctly implemented and did not produce any results. Therefore, it is inconclusive whether the conjecture is supported or falsified. | next: Re-implement the test code to fix the NameError and run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12966
2	preregistration	ollama_remote	glm4:latest	0	0	6459
3	novelty	ollama_remote	glm4:latest	0	0	4662
4	novelty	ollama_remote	glm4:latest	0	0	5910
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42211
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13140
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10249
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10723
9	judge	ollama_remote	glm4:latest	0	0	55303

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 161624 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7ba1444f0780.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7ba1444f0780.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7ba1444f0780.tar.gz` (if generated)